SESSION DE RATTRAPAGE – JEE1 ET JEE2

Document d'explication

Conception et fonctionnement de l'application CRUD de facturation

Clavstel El Schaddai BOYAOU

Classe : M1 GELO

TP1 : Spring Boot + HTML/CSS + MySQL

Année universitaire 2025-2026

1. Introduction

Ce document explique la conception et le fonctionnement de l'application de facturation réalisée pour le TP1 de JEE. Il s'agit d'une application web **CRUD** (Create, Read, Update, Delete) développée avec **Spring Boot**, dont les pages sont écrites en **HTML et CSS** (moteur Thymeleaf) et dont les données sont stockées dans **MySQL**. Le gérant s'authentifie puis gère les clients et les factures.

2. Cahier des charges

- Entités : Facture (Idfact, montant, objet, IdClient), Client (Idclient, nom, tel), Gérant (Id, nom, login, password).
- Technologies : Spring Boot, HTML, CSS, MySQL.
- Fonctionnalités : authentification du gérant ; ajout, modification, suppression et recherche des clients et des factures.

3. Architecture MVC

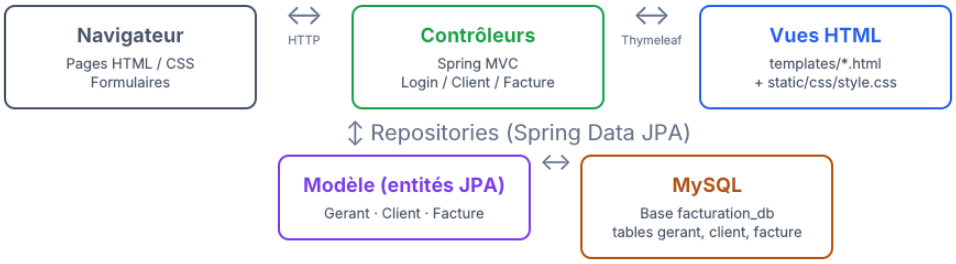


Figure 1 – Architecture de l'application

L'application suit le modèle **MVC** (Modèle – Vue – Contrôleur) :

- **Modèle :** les entités JPA (Gerant, Client, Facture) et les repositories qui les lisent ou les écrivent dans MySQL.
- **Vue :** les modèles HTML Thymeleaf (dossier templates) mis en forme par la feuille CSS style.css.
- **Contrôleur :** les classes annotées @Controller qui reçoivent les requêtes HTTP, appellent les repositories et désignent la vue à afficher.

Organisation du projet

Élément	Rôle
model/	Entités JPA : Gerant, Client, Facture (annotations @Entity, @ManyToOne, @OneToMany, validation)
repository/	Interfaces JpaRepository : opérations CRUD et requêtes de recherche
controller/	LoginController, ClientController, FactureController
security/	GerantUserDetailsService : charge le gérant depuis la base pour Spring Security
config/	SecurityConfig (règles d'accès, BCrypt) et DataInitializer (données de démonstration)
templates/	Pages HTML Thymeleaf : login, clients, client-form, factures, facture-form, fragments/layout
static/css/	Feuille de style style.css

Élément	Rôle
database/	Script SQL facultatif de création des tables

4. Base de données MySQL



Figure 2 – Tables MySQL

Les trois tables sont créées automatiquement par Hibernate (`spring.jpa.hibernate.ddl-auto=update`) ; le script `database/facturation.sql` donne l'équivalent manuel. La relation entre client et facture est une **clé étrangère** : `facture.idclient` référence `client.idclient`. Côté Java, c'est un `@ManyToOne` dans Facture et un `@OneToMany(mappedBy="client", cascade=REMOVE)` dans Client : supprimer un client supprime ses factures.

5. Sécurité

- **Authentification :** Spring Security affiche la page /login. `GerantUserDetailsService` retrouve le gérant par son login ; le mot de passe saisi est comparé au hash **BCrypt** de la base. Aucun mot de passe n'est stocké en clair.
- **Autorisation :** seules les URL /login et /css/** sont publiques ; toutes les autres exigent le rôle GERANT. Un visiteur non connecté est redirigé vers /login.
- **CSRF :** chaque formulaire POST (enregistrer, supprimer, déconnexion) inclut un jeton ajouté automatiquement par Thymeleaf.
- **Session :** après connexion, l'identité est conservée dans la session HTTP du serveur ; la déconnexion l'invalide.

6. Détail des fonctionnalités

6.1 Gestion des clients (ClientController)

- **Lister / rechercher :** GET /clients?q=… ; sans terme, tous les clients triés par nom ; sinon recherche par nom (insensible à la casse) ou téléphone.
- **Ajouter :** GET /clients/nouveau affiche le formulaire ; POST /clients/save valide (nom obligatoire, téléphone au bon format) puis enregistre.
- **Modifier :** GET /clients/{id}/modifier pré-remplit le même formulaire ; à l'enregistrement, seuls nom et téléphone sont recopiés sur l'entité existante.
- **Supprimer :** POST /clients/{id}/supprimer après confirmation JavaScript ; les factures du client sont supprimées par cascade.

6.2 Gestion des factures (FactureController)

- **Lister / rechercher :** requête JPQL qui cherche dans l'objet ou le nom du client ; tri par numéro décroissant.

- **Ajouter / modifier :** le client est choisi dans une liste déroulante ; le contrôleur vérifie qu'il existe, valide le montant (positif) et l'objet (non vide).
- **Supprimer :** POST /factures/{id}/supprimer après confirmation.

6.3 Messages et validation

Après chaque opération, un message de confirmation vert (attribut flash) s'affiche. Les erreurs de validation sont affichées en rouge sous les champs concernés ; le formulaire n'est pas enregistré tant qu'elles ne sont pas corrigées.

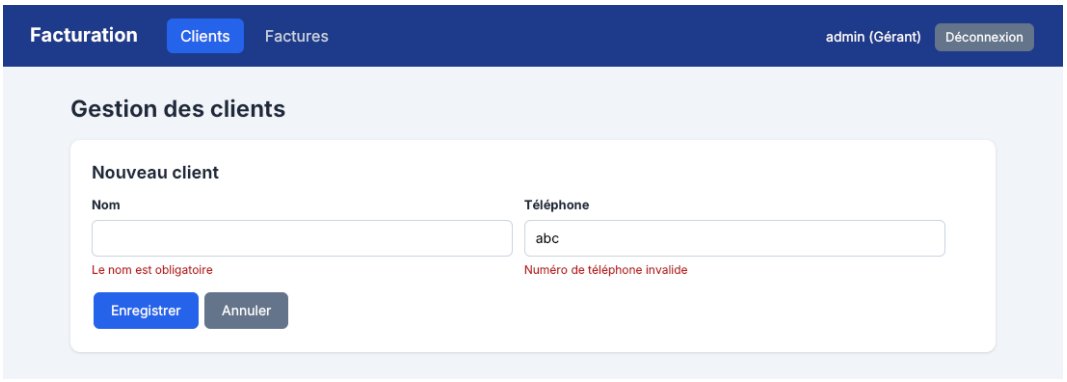


Figure 3 – Erreurs de validation

7. Déroulement d'une requête (ajout d'une facture)

- Le gérant clique sur « + Nouvelle facture » : GET /factures/nouveau, le contrôleur renvoie la vue facture-form avec la liste des clients.
- Il remplit le formulaire et valide : POST /factures/save (avec jeton CSRF).
- Spring Security vérifie la session et le rôle ; FactureController valide les données et charge le client choisi.
- Le repository enregistre la facture dans MySQL (INSERT).
- Redirection vers /factures avec le message « Facture ajoutée avec succès ».

8. Mise en route et tests

Voir le guide d'utilisation. Scénarios de test recommandés : connexion avec un mauvais mot de passe, accès direct à /clients sans être connecté (redirection), ajout d'un client invalide (erreurs), ajout puis modification d'un client, ajout d'une facture, recherche, suppression d'un client (ses factures disparaissent), déconnexion.

9. Limites et améliorations possibles

- Pas de tests automatisés (JUnit, MockMvc) : à ajouter.
- Mot de passe MySQL vide et gérant admin/admin123 : à changer avant tout usage réel.
- Pas de pagination des listes.
- Le client ne peut pas se connecter : c'est l'objet du **TP2**, qui remplace aussi HTML/CSS par React et MySQL par MongoDB.